

算法都在名字后面附加一个 `_copy` (参见 10.2.2 节, 第 341 页):

```
reverse(beg, end);           // 反转输入范围中元素的顺序
reverse_copy(beg, end, dest); // 将元素按逆序拷贝到 dest
```

一些算法同时提供 `_copy` 和 `_if` 版本。这些版本接受一个目的位置迭代器和一个谓词:

```
// 从 v1 中删除奇数元素
remove_if(v1.begin(), v1.end(),
          [](int i) { return i % 2; });
// 将偶数元素从 v1 拷贝到 v2; v1 不变
remove_copy_if(v1.begin(), v1.end(), back_inserter(v2),
               [](int i) { return i % 2; });
```

两个算法都调用了 `lambda` (参见 10.3.2 节, 第 346 页) 来确定元素是否为奇数。在第一个调用中, 我们从输入序列中将奇数元素删除。在第二个调用中, 我们将非奇数 (亦即偶数) 元素从输入范围拷贝到 `v2` 中。

10.5.3 节练习

415

练习 10.41: 仅根据算法和参数的名字, 描述下面每个标准库算法执行什么操作:

```
replace(beg, end, old_val, new_val);
replace_if(beg, end, pred, new_val);
replace_copy(beg, end, dest, old_val, new_val);
replace_copy_if(beg, end, dest, pred, new_val);
```

10.6 特定容器算法

与其他容器不同, 链表类型 `list` 和 `forward_list` 定义了几个成员函数形式的算法, 如表 10.6 所示。特别是, 它们定义了独有的 `sort`、`merge`、`remove`、`reverse` 和 `unique`。通用版本的 `sort` 要求随机访问迭代器, 因此不能用于 `list` 和 `forward_list`, 因为这两个类型分别提供双向迭代器和前向迭代器。

链表类型定义的其他算法的通用版本可以用于链表, 但代价太高。这些算法需要交换输入序列中的元素。一个链表可以通过改变元素间的链接而不是真的交换它们的值来快速“交换”元素。因此, 这些链表版本的算法的性能比对应的通用版本好得多。

Best Practices

对于 `list` 和 `forward_list`, 应该优先使用成员函数版本的算法而不是通用算法。

表 10.6: <code>list</code> 和 <code>forward_list</code> 成员函数版本的算法	
这些操作都返回 <code>void</code>	
<code>lst.merge(lst2)</code>	将来自 <code>lst2</code> 的元素合并入 <code>lst</code> 。 <u><code>lst</code> 和 <code>lst2</code> 都必须是有顺序的。</u>
<code>lst.merge(lst2, comp)</code>	元素将从 <code>lst2</code> 中删除。在合并之后, <code>lst2</code> 变为空。第一个版本使用 <code><</code> 运算符; 第二个版本使用给定的比较操作
<code>lst.remove(val)</code>	调用 <code>erase</code> 删除掉与给定值相等 (<code>==</code>) 或令一元谓词为真的每个元素
<code>lst.remove_if(pred)</code>	
<code>lst.reverse()</code>	反转 <code>lst</code> 中元素的顺序

续表

<code>lst.sort()</code>	使用<或给定比较操作排序元素
<code>lst.sort(comp)</code>	
<code>lst.unique()</code>	调用 <code>erase</code> 删除同一个值的连续拷贝。第一个版本使用==；第
<code>lst.unique(pred)</code>	二个版本使用给定的二元谓词

 splice 成员

416

链表类型还定义了 `splice` 算法，其描述见表 10.7。此算法是链表数据结构所特有的，因此不需要通用版本。

表 10.7: <code>list</code> 和 <code>forward_list</code> 的 <code>splice</code> 成员函数的参数	
<code>lst.splice(args)</code> 或 <code>flst.splice_after(args)</code>	
<code>(p, lst2)</code>	<code>p</code> 是一个指向 <code>lst</code> 中元素的迭代器，或一个指向 <code>flst</code> 首前位置的迭代器。函数将 <code>lst2</code> 的所有元素移动到 <code>lst</code> 中 <code>p</code> 之前的位置或是 <code>flst</code> 中 <code>p</code> 之后的位置。将元素从 <code>lst2</code> 中删除。 <code>lst2</code> 的类型必须与 <code>lst</code> 或 <code>flst</code> 相同，且不能是同一个链表
<code>(p, lst2, p2)</code>	<code>p2</code> 是一个指向 <code>lst2</code> 中位置的有效迭代器。将 <code>p2</code> 指向的元素移动到 <code>lst</code> 中，或将 <code>p2</code> 之后的元素移动到 <code>flst</code> 中。 <code>lst2</code> 可以是与 <code>lst</code> 或 <code>flst</code> 相同的链表
<code>(p, lst2, b, e)</code>	<code>b</code> 和 <code>e</code> 必须表示 <code>lst2</code> 中的合法范围。将给定范围中的元素从 <code>lst2</code> 移动到 <code>lst</code> 或 <code>flst</code> 。 <code>lst2</code> 与 <code>lst</code> (或 <code>flst</code>) 可以是相同的链表，但 <code>p</code> 不能指向给定范围中元素

链表特有的操作会改变容器

多数链表特有的算法都与其通用版本很相似，但不完全相同。链表特有版本与通用版本间的一个至关重要的区别是链表版本会改变底层的容器。例如，`remove` 的链表版本会删除指定的元素。`unique` 的链表版本会删除第二个和后继的重复元素。

类似的，`merge` 和 `splice` 会销毁其参数。例如，通用版本的 `merge` 将合并的序列写到一个给定的目的迭代器；两个输入序列是不变的。而链表版本的 `merge` 函数会销毁给定的链表——元素从参数指定的链表中删除，被合并到调用 `merge` 的链表对象中。在 `merge` 之后，来自两个链表中的元素仍然存在，但它们都已在同一个链表中。

10.6 节练习

练习 10.42: 使用 `list` 代替 `vector` 重新实现 10.2.3 节（第 343 页）中的去除重复单词的程序。

小结

417

标准库定义了大约 100 个类型无关的对序列进行操作的算法。序列可以是标准库容器类型中的元素、一个内置数组或者是（例如）通过读写一个流来生成的。算法通过在迭代器上进行操作来实现类型无关。多数算法接受的前两个参数是一对迭代器，表示一个元素范围。额外的迭代器参数可能包括一个表示目的位置的输出迭代器，或是表示第二个输入范围的另一个或另一对迭代器。

根据支持的操作不同，迭代器可分为五类：输入、输出、前向、双向以及随机访问迭代器。如果一个迭代器支持某个迭代器类别所要求的操作，则属于该类别。

如同迭代器根据操作分类一样，传递给算法的迭代器参数也按照所要求的操作进行分类。仅读取序列的算法只要求输入迭代器操作。写入数据到目的位置迭代器的算法只要求输出迭代器操作，依此类推。

算法从不直接改变它们所操作的序列的大小。它们会将元素从一个位置拷贝到另一个位置，但不会直接添加或删除元素。

虽然算法不能向序列添加元素，但插入迭代器可以做到。一个插入迭代器被绑定到一个容器上。当我们将一个容器元素类型的值赋予一个插入迭代器时，迭代器会将该值添加到容器中。

容器 `forward_list` 和 `list` 对一些通用算法定义了自己特有的版本。与通用算法不同，这些链表特有版本会修改给定的链表。

术语表

back_inserter 这是一个迭代器适配器，它接受一个指向容器的引用，生成一个插入迭代器，该插入迭代器用 `push_back` 向指定容器添加元素。

双向迭代器 (bidirectional iterator) 支持前向迭代器的所有操作，还具有用 `--` 在序列中反向移动的能力。

二元谓词 (binary predicate) 接受两个参数的谓词。

bind 标准库函数，将一个或多个参数绑定到一个可调表达式。`bind` 定义在头文件 `functional` 中。

可调对象 (callable object) 可以出现在调用运算符左边的对象。函数指针、`lambda` 以及重载了函数调用运算符的类的对象都是可调对象。

捕获列表 (capture list) `lambda` 表达式的

一部分，指出 `lambda` 表达式可以访问所在上下文中哪些变量。

cref 标准库函数，返回一个可拷贝的对象，其中保存了一个指向不可拷贝类型的 `const` 对象的引用。

前向迭代器 (forward iterator) 可以读写元素，但不必支持 `--` 的迭代器。

front_inserter 迭代器适配器，给定一个容器，生成一个用 `push_front` 向容器开始位置添加元素的插入迭代器。

泛型算法 (generic algorithm) 类型无关的算法。

输入迭代器 (input iterator) 可以读但不能写序列中元素的迭代器。

插入迭代器 (insert iterator) 迭代器适配器，生成一个迭代器，该迭代器使用容器操作向给定容器添加元素。

418